

TraceFlow Compress - Project Report

An LLM Governance & Token-Optimization Gateway

Live product: <https://traceflow-compress.vercel.app>

MCP connector: <https://traceflow-compress.vercel.app/mcp>

Source code: <https://github.com/ashritkvs/traceflow-compress>

Date: July 2026

1. Executive summary

TraceFlow Compress is a working, deployed **gateway that sits between an application (or AI agent) and a Large Language Model. Before any prompt reaches the LLM, the gateway classifies it, screens it for security and safety, optimizes it to use fewer tokens, verifies meaning is preserved, and records full cost/carbon/usage metrics - returning a single combined result.**

It was built to realize two source specifications at once: the **TraceFlow** whitepaper (a token-aware observability platform) and the **TokenOps** brief (an API-based token-utilization observability & orchestration solution). The delivered product covers **every functional item of the TokenOps spec** and roughly **70% of the TraceFlow whitepaper's substance** - the portion that does not require GPU hardware or a live inference-serving stack.

The product is live on the public internet, exposed both as a **human web tool** and as a **20-tool MCP connector** for AI agents, with a multi-tenant SaaS shell (API keys, rate limiting, plan tiers, usage metering). It has 34 automated tests, a clean static-security scan (Semgrep, 0 findings), and a measured benchmark.

2. Background - the two source specs

TraceFlow (whitepaper). A conceptual design for a "token-aware observability" platform that makes the **token** - not the request - the unit of measurement for LLM workloads, correlating token usage with compute, latency, cost, energy, and carbon, and adding a "prompt intelligence" layer for redundant-token detection and compression recommendations.

TokenOps (brief). An API-based solution for token-utilization observability and orchestration, specified as nine functional requirements (token accounting, prompt classification, a prompt knowledge-base, prompt optimization, content security, non-vulnerable code, non-violent content, violation alerting, LLM integration) plus five design aspects (architecture, infrastructure, connector,

marketplace, and code-package policy).

TraceFlow Compress unifies both into one product: TraceFlow supplies the **observability + compression** half, TokenOps supplies the **governance + orchestration half**.

3. What was built - product overview

Every prompt passes through a single pipeline (one API call / one MCP tool):

Prompt

```
-> GOVERN    classify -> security scan -> moderation -> package policy -> code scan
              -> verdict: ALLOW / WARN / BLOCK    (blocked prompts stop here)
-> COMPRESS  heuristic (free) - adaptive/LLM (quality) - safe (verified)
-> VERIFY    meaning score 0-100  (optional)
-> MEASURE   tokens - cost - carbon - GPU-load - latency  (+ trace spans)
-> METER     per-tenant usage vs plan quota
```

Result: { verdict, prompt type, compressed_text, full metrics, governance }

It is delivered three ways: an **interactive web dashboard** (paste a prompt, see the result and metrics), a **REST API** (`/process``, `/compress``, `/metrics``, ...), and a **Model Context Protocol (MCP) connector** (`/mcp``) that any AI agent can call as 20 tools.

A deliberate **credit-safe access model** underpins it: anonymous users get the free, local, rule-based tier (no API cost); a valid API key unlocks the paid LLM tier and the connector.

4. TokenOps spec - detailed coverage

1. Total tokens used / available. Built. The gateway measures every prompt's token count exactly (via tiktoken) and aggregates usage. A **budget module** (`/get_budget``) reports tokens used, tokens available against a configured monthly budget, utilization percentage, and - uniquely - how far compression **stretches** that budget. **Caveat:** "available" is measured against a configured budget, not a live provider-account quota (which would require the provider's billing API).

2. Prompt types by output. Built. The `/classify_prompt`` capability labels each prompt as **enquiry, code, testing, draft, ppt, or refinement** using a keyword/pattern classifier, with a confidence score. This drives reporting and could drive routing/policy.

3. Prompt knowledge base / avoid re-run. Built. A **semantic cache** stores seen prompts and returns a prior result for exact repeats **and** near-duplicates (lexical-cosine similarity, with an optional embedding mode), so identical or

similar prompts are never recomputed. Namespaced by settings so a cache hit is always valid.

4. Prompt optimization for token utilization. Built - the core. Compression runs in four modes: a **free heuristic** (rule-based filler/redundancy removal + token-importance scoring), an **LLM quality** mode (rewrites the prompt), an **adaptive** mode (uses the free tier and escalates to the LLM only when meaning drops), and a **safe** mode (verifies and falls back to the original rather than ship a broken compression). Measured average reduction is ~41%.

5. Content security checks. Built. ``scan_content`` detects **PII** (emails, phones, SSNs, credit cards), **secrets** (API keys, AWS keys, private keys, hard-coded tokens), and **prompt-injection** attempts ("ignore previous instructions", role-hijack, jailbreak patterns), assigning a risk level that can block the prompt. *Caveat:* regex/heuristic - effective on common cases, not an audited guarantee.

6. Non-vulnerable code. Built (as a scanner). ``scan_code`` flags dangerous code patterns - ``eval`/`exec``, ``os.system``, ``subprocess(shell=True)``, ``pickle.loads``, SQL-injection f-strings, hard-coded secrets, weak hashing, disabled TLS verification - and can block on high risk. *Caveat:* it **flags** risky code; it does not generate code or guarantee safety.

7. Non-violent content. Built. A **moderation** step flags violence, hate, self-harm and related categories, using the OpenAI Moderation API when a key is present (free) and a keyword heuristic otherwise.

8. Alert mechanism for violations. Built (core). Every warn/block verdict is recorded as a **violation event**, retrievable via ``get_violations`` and the ``/violations`` endpoint, and surfaced on the dashboard. *Caveat:* external delivery (webhook/Slack/email) is not yet wired.

9. LLM integration feasibility. Built (as feasibility + connector). ``list_providers`` documents integration with OpenAI, Anthropic, Google, Mistral, and local/Ollama models and reports which are configured; the gateway itself is **provider-agnostic** and callable by any MCP client. *Caveat:* only OpenAI is actually *called* today for the LLM features.

Design aspect - architecture. Documented (``SPEC.md``, ``PRODUCT.md``): a layered design of core library -> governance orchestrator -> unified pipeline -> MCP server -> serverless HTTP -> dashboards.

Design aspect - infrastructure. Built: deployed serverless on Vercel (Fluid Compute, Python), scaling to zero, with a metrics store that runs on a local file or Upstash Redis. *Caveat:* durable persistence needs Upstash keys.

Design aspect - convert to connector. Done. It is a live, streamable-HTTP

MCP connector exposing 20 tools plus a metrics resource, addable to Claude or any MCP client.

Design aspect - marketplace. Partially: a machine-readable **manifest** (`/manifest`) and a `MARKETPLACE.md` submission checklist exist; formal listing (OAuth, privacy policy, billing) is future work.

Design aspect - package policy for code agents. Built. `check_policy` detects package references (`import / pip / npm / require`) and evaluates them against a configurable **allowlist or denylist**; the governance layer blocks prompts that pull disallowed dependencies.

Net: all nine functional requirements and all five design aspects are addressed - six fully, the remainder with clearly stated caveats.

5. TraceFlow whitepaper - coverage (~70%)

Built: the token-centric premise; token/cost/energy/carbon math; a "prompt intelligence" layer (redundancy detection, compression, most-compressible prompts); **distributed-trace spans** (measured sub-step timings); correlation of tokens to cost/energy/carbon/compute-load; AIOps **anomaly** detection **and** optimization (**trim/route/cache**); executive and engineering dashboards; **multi-model** routing; semantic caching; **and** savings **forecasting**.

Replaced honestly: the whitepaper's GPU/VRAM telemetry cannot be measured without GPU hardware, so it is replaced by an **estimated compute-load model** (`2 x params x tokens` -> GPU-milliseconds) plus **real CPU/RAM measurement** - every estimate is explicitly labeled, and closed-model parameter counts are flagged as assumptions.

Not built (blocked by hardware/traffic, not effort): real GPU/VRAM/KV-cache telemetry, passive interception of live inference traffic, and token-aware autoscaling of physical infrastructure.

6. Architecture & infrastructure

Stack: Python 3.11 - FastMCP (Model Context Protocol, streamable HTTP) - Starlette (ASGI) - tiktoken - deployed on Vercel serverless (Fluid Compute).

Modules (~18 core files): compression, prompt intelligence, estimation, compute-load, cache, routing, verification, governance (classification, security, moderation, policy, code-scan), unified pipeline, metrics store, budget, plans/metering, rate limiting, forecasting, anomaly detection, provider

registry, manifest.

Interfaces: interactive web dashboard + engineering dashboard; REST API (`/process``, `/compress``, `/metrics``, `/records``, `/violations``, `/usage``, `/manifest``); MCP connector (`/mcp``).

SaaS shell: multi-tenant API keys, per-identity rate limiting (Upstash or in-memory), plan tiers (free/pro/enterprise), usage metering, and monthly quota enforcement.

Security posture: static analysis clean (Semgrep, 0 findings); secrets kept in environment variables (never committed); the compute endpoints are API-key-gated with anonymous access limited to the free, no-cost tier.

7. Results & metrics

- **20 MCP tools, 34 automated tests passing, Semgrep 0 findings.**
 - **Benchmark (32-prompt labeled set):** ~41% average token reduction at ~1 ms; 100% governance detection and 0% false-positives *on that set*.
 - **Live-verified:** compression, governance blocking, classification, dashboards, the connector handshake, auth gating, and rate limiting all confirmed on the production URL.
-

8. Honest limitations

To keep any presentation credible, these are stated plainly:

1. The free compressor is **rule-based** and produces choppier output than a trained ML compressor; the LLM tier is the quality path.
 2. The benchmark is a **small, self-authored set** - cite it as "on our N-prompt test set," not as generalized accuracy.
 3. Safety detectors are **heuristic** (regex/keyword + optional moderation API): effective on common cases, **not an independently audited enterprise** guarantee.
 4. Cost, energy, carbon, and GPU-load figures are **estimates**, clearly labeled; closed-model parameters are assumed.
 5. Metrics/usage **persistence** needs Upstash (currently ephemeral on serverless); billing is **metering only** (no payment processor connected).
-

9. Roadmap

Near-term (code): durable persistence (Upstash); a public landing page; a tenant self-serve usage dashboard; a larger, adversarial benchmark; Stripe

billing; real multi-provider calls; external violation alerts (webhook/Slack).

Beyond code: an independent security audit; a trained ML compressor to match best-in-class quality; and go-to-market/distribution. These require professionals, data/compute, and users respectively - not further engineering alone.

Prepared for demonstration. All figures are measured or clearly-labeled estimates; nothing in this report is fabricated. The product is live and can be exercised immediately at the links above.